

# **FLEXYGO RAG System**

## **Documentación de Componentes**

Generado: 16/06/2026 11:55

Sistema de Búsqueda Aumentada por Recuperación (RAG) genérico para AHORA

## Tabla de Contenidos

1. RAGConfig.cs - Configuración del RAG
2. RAGDocument.cs - Estructura de Documentos
3. RAGVector.cs - Vectores y Embeddings
4. VectorMath.cs - Operaciones Matemáticas
5. RAGProcessor.cs - Búsqueda de Similitud
6. WebSearch.cs - Búsqueda en Internet
7. AssistantCalling.cs - Encadenamiento de Asistentes
8. RAGCategory.cs - Categorización de Documentos

## Introducción

Este es un sistema RAG (Retrieval-Augmented Generation) genérico integrado en Flexygo. Permite que cualquier cliente de AHORA pueda indexar documentos propios, realizar búsquedas semánticas sobre ellos, y obtener respuestas mejoradas usando IA.

**Caso de uso genérico:** Un cliente sube N documentos (PDFs, manuales, reportes) a través de Flexygo. El RAG los convierte a embeddings, los almacena, y luego cuando el usuario hace una pregunta en chat, busca los documentos más relevantes y los pasa a un LLM para obtener una respuesta contextualizada.

## 1. RAGConfig.cs - Configuración del RAG

Define todos los parámetros de configuración del RAG. Controla cómo se generan embeddings, los umbrales de similitud, y qué propiedades de los documentos son buscables.

### Propiedades principales:

| Propiedad             | Tipo   | Propósito                                              |
|-----------------------|--------|--------------------------------------------------------|
| RagId                 | string | Identificador único del RAG (por cliente o proyecto)   |
| EmbeddingModel        | string | Modelo a usar (ej: text-embedding-3-small de OpenAI)   |
| EmbeddingDimensions   | int    | Dimensiones del vector (típicamente 1536)              |
| EmbeddingContext      | int    | Máximo de tokens que acepta el modelo embedding        |
| VectorThreshold       | float  | Umbral de similitud para filtrar resultados relevantes |
| ObjectName            | string | Nombre del objeto que contiene los documentos          |
| SearchProperties      | string | Qué campos de los documentos se pueden buscar          |
| FullDocumentEmbedding | bool   | Si embeber el doc completo o fragmentarlo              |
| MinWordsPerSegment    | int    | Mínimo de palabras por fragmento                       |

### Código del archivo:

```
public class RAGConfig { public string RagId { get; set; } public string ConfigId { get; set; } public string EmbeddingModel { get; set; } public int EmbeddingDimensions { get; set; } public float VectorThreshold { get; set; } public bool FullDocumentEmbedding { get; set; } public int MinWordsPerSegment { get; set; } public string ObjectName { get; set; } }
```

### ¿Cómo se usa en Flexygo?

Cada cliente/proyecto en Flexygo puede tener su propia RAGConfig. Define si quiere usar OpenAI embeddings, qué threshold de similitud usar, si fragmentar documentos largos, etc. Se almacena en BD y se carga cuando se inicializa el RAG.

### ■ Issues/Mejoras:

- El threshold está hardcodedo en RAGProcessor (0.34), debería leerlo de RAGConfig
- Las dimensiones (1536) están hardcodedas en VectorMath, deberían venir de RAGConfig
- No hay validación de que el EmbeddingModel exista o sea accesible
- Falta versionado de configs (un cliente podría tener múltiples versiones)

## 2. RAGDocument.cs - Estructura de Documentos

Representa un documento individual que se ha ingestado en el RAG. Contiene el contenido de texto, su embedding vectorial, y metadatos para rastreabilidad y búsqueda.

### Propiedades principales:

| Propiedad      | Tipo    | Propósito                                              |
|----------------|---------|--------------------------------------------------------|
| RagId          | string  | ID del RAG al que pertenece                            |
| DocumentId     | string  | Identificador único del documento                      |
| Reference      | string  | Link al documento original en la BD del cliente        |
| TextContent    | string  | Contenido de texto que se busca                        |
| VectorContent  | float[] | Embedding del documento (1536 dimensiones típicamente) |
| Type           | string  | Categoría/tipo de documento (ej: manual, reporte, etc) |
| Link           | string  | URL o referencia dentro de Flexygo                     |
| WarningMessage | string  | Flags de errores durante la ingesta                    |

### Código del archivo:

```
public class RAGDocument { public string RagId { get; set; } public string DocumentId { get; set; } public string Reference { get; set; } public string TextContent { get; set; } public float[] VectorContent { get; set; } public string Type { get; set; } public string Link { get; set; } public string WarningMessage { get; set; } }
```

### ¿Cómo se usa en Flexygo?

Cuando un cliente sube un documento a través de Flexygo (PDF, Word, etc), el sistema crea un RAGDocument. Lo procesa (extrae texto, lo fragmenta si es muy largo), calcula su embedding con OpenAI, y lo almacena. Luego, cuando busca, compara queryEmbedding vs todos los VectorContent.

### ■ Issues/Mejoras:

- No hay validación de que VectorContent tenga la dimensión correcta
- TextContent no se fragmenta automáticamente para docs largos
- Falta timestamp de ingesta (cuándo se subió el doc)
- Falta información de usuario que subió el documento (para auditoría)
- No hay soporte para metadatos key-value flexibles

### 3. RAGVector.cs - Vectores y Embeddings

Representa un vector de embedding junto con su score de similitud. Es lo que RAGProcessor devuelve cuando encuentra documentos relevantes para una query.

#### Propiedades principales:

| Propiedad  | Tipo    | Propósito                                        |
|------------|---------|--------------------------------------------------|
| RagId      | string  | ID del RAG                                       |
| DocumentId | string  | ID del documento de origen                       |
| Embedding  | float[] | Array de N dimensiones (1536 típicamente)        |
| Score      | float   | Similitud con la query (debería estar entre 0-1) |

#### Código del archivo:

```
public class RAGVector { public string RagId { get; set; } public string DocumentId { get; set; } public float[] Embedding { get; set; } public float Score { get; set; } }
```

#### ¿Cómo se usa en Flexygo?

Cuando un usuario escribe una pregunta, Flexygo convierte la query a embedding (con OpenAI). Luego RAGProcessor crea una lista de RAGVector comparando la queryEmbedding con todos los documentos. Devuelve los top-N RAGVector con mayor Score (más similares).

#### ■ ■ Issues/Mejoras:

- Score no está normalizado (es dot product directo, rango -1536 a +1536)
- Debería normalizarse a 0-1 para ser interpretable
- No hay información de qué método de similitud se usó
- Falta timestamp de búsqueda

## 4. VectorMath.cs - Operaciones Matemáticas

Utilidades estáticas para operaciones vectoriales. Proporciona métodos para comparar embeddings usando diferentes métricas de similitud.

### Métodos disponibles:

| Método                  | Fórmula                                | Rango                          |
|-------------------------|----------------------------------------|--------------------------------|
| DotProduct(a, b)        | $\sum a[i] \times b[i]$                | -1536 a +1536 (sin normalizar) |
| CosineSimilarity(a, b)  | $\text{DotProduct} / ( a  \times  b )$ | 0 a 1 (normalizado)            |
| EuclideanDistance(a, b) | $\sqrt{\sum (a[i] - b[i])^2}$          | 0 a infinito                   |
| Length(v)               | $\sqrt{\sum v[i]^2}$                   | Magnitud del vector            |

### Código del archivo (fragmento):

```
public static class VectorMath { private const int VectorDimension = 1536; public
static float DotProduct(float[] a, float[] b) { float sum = 0; for (int i = 0; i <
VectorDimension; i++) sum += a[i] * b[i]; return sum; } public static float
CosineSimilarity(float[] a, float[] b) { // ■■ INCORRECTO: debería ser DotProduct /
(Length(a) * Length(b)) return DotProduct(a, b) / (float)Math.Pow(VectorDimension, 2);
} }
```

### ¿Cómo se usa en Flexygo?

RAGProcessor usa VectorMath.DotProduct() en su bucle principal para comparar la query embedding con cada documento almacenado. Los resultados se filtran por un threshold.

#### ■ CRÍTICO - Bug matemático:

##### CosineSimilarity está mal implementado.

Divide por  $\text{VectorDimension}^2$  en lugar de  $\text{Length}(a) \times \text{Length}(b)$ .

La fórmula correcta es:  $(a \cdot b) / (|a| \times |b|)$

Impacto: Si alguien usa este método, obtendrá valores incorrectos.

#### ■■ Otros issues:

- VectorDimension (1536) está hardcodedo, debería venir de RAGConfig
- DotProduct no normaliza — devuelve valores en rango -1536 a +1536
- Sin validación de dimensiones antes de operar
- Sin manejo de vectores NaN o infinitos

## 5. RAGProcessor.cs - Búsqueda de Similitud

El motor de búsqueda del RAG. Procesa una query embedding contra un dataset de vectores almacenados y devuelve los documentos más relevantes usando búsqueda paralela.

### Método principal:

```
ParallelSearchByDotProductThreshold(RAGVector query, int chunkSize, int limit = 5)
```

### Parámetros:

- query: Vector de búsqueda (query embedding convertido a RAGVector)
- chunkSize: Tamaño de chunks para paralelización (ej: 100)
- limit: Máximo de resultados a devolver (default 5)

### Flujo de búsqueda:

```
1. Valida que query sea de 1536 dimensiones 2. Divide el dataset en chunks de tamaño configurable 3. Para cada chunk, crea un Task paralelo que: a. Itera sobre cada vector en el chunk b. Calcula DotProduct(query, documento) c. Si score >= threshold (0.34), añade a resultados 4. Espera Task.WaitAll() a que todos terminen 5. Ordena resultados por Score descendente 6. Devuelve top 'limit' resultados
```

### ¿Cómo se usa en Flexygo?

Flujo completo en Flexygo:

1. Usuario escribe pregunta en chat
2. Flexygo convierte query a embedding (OpenAI API)
3. Crea RAGVector con ese embedding
4. Llama a RAGProcessor.ParallelSearchByDotProductThreshold()
5. Obtiene top-5 documentos similares
6. Pasa esos documentos + query a GPT-4o
7. GPT-4o genera respuesta contextualizada

### ■ Issues críticos:

- **Threshold 0.34 es muy bajo:** DotProduct sin normalizar va de -1536 a +1536. Un threshold de 0.34 aceptará casi cualquier documento.
- **Búsqueda O(n):** Sin HNSW en SQL Server, compara contra TODOS los documentos. Con 100k docs es lento. Con 1M es muy lento.
- **Sin índices:** Cada búsqueda rescanea la tabla completa.
- **Sin caché:** La misma query se busca N veces = N cálculos redundantes.
- **Sin timeout:** Task.WaitAll() puede colgar indefinidamente.

### ■■ Otros issues:

- Chunk size no se optimiza según cores disponibles
- Sin logging de métricas (hits, promedio score, tiempo)
- Sin monitoreo de performance

## 6. WebSearch.cs - Búsqueda en Internet

Permite buscar en Google Custom Search API y extraer contenido de páginas web. Se usa como alternativa o complemento cuando el RAG no tiene documentos relevantes.

### Método principal:

```
SearchInWeb(AhoraMind mind, ChatGPTAssistant assistant, ChatGPTTool tool, int loopCounter, string toolId, List messages)
```

### Flujo:

```
1. Valida que Google API key y Search Engine ID estén configurados 2. Construye URL: customsearch.googleapis.com/v1?key=X&cx=Y&q=Z 3. Envía GET request 4. Para cada resultado (máximo 3): a. Abre URL con Playwright (headless) b. Espera DOMContentLoaded c. Extrae document.body.innerText d. Filtra PDFs 5. Devuelve contenido combinado al usuario
```

### ¿Cómo se usa en Flexygo?

Si RAGProcessor no encuentra documentos relevantes, o el usuario pide 'buscar en Internet', WebSearch toma la query, busca en Google, y extrae el contenido de las top-3 páginas. Ese contenido se pasa a GPT-4o para responder.

### ■ ■ Issues importantes:

- **Playwright lento:** ~2-5s por página en headless. Con 3 páginas = 6-15s.
- **Sin pool de conexiones:** Crea nuevo IBrowser cada vez (despilfarro).
- **Async/await incorrecto:** Usa await ... .Result (antipatrón).
- **Sin timeout:** page.GotoAsync() puede colgar indefinidamente.
- **Sin caché:** La misma búsqueda se repite = múltiples hits a Google.
- **Dependencia de Edge:** Requiere Microsoft Edge, no Chrome.
- **Extracción cruda:** document.body.innerText puede traer código JS, CSS, etc.

## 7. AssistantCalling.cs - Encadenamiento de Asistentes

Permite que un asistente delegue preguntas a otros asistentes especializados. Soporta cadenas de llamadas con detección básica de loops infinitos.

### Método principal:

```
CallAssistant(ChatGPTAssistant asstnt, ChatGPTTool tool, int counter, string toolId, List messages, AhoraMind mind, bool copyAnswer = true)
```

### Flujo:

```
1. Lee assistantId y assistantPrompt del tool.Parameters 2. Valida que no haya loop (mind.AssistantsList.Count < 30) 3. Para cada assistantId (separados por |): a. Inyecta prompt en el último mensaje del usuario b. Llama a mind.ChatAsync() con el asistente remoto c. Obtiene respuesta d. Si copyAnswer=true, acumula respuesta; si no, pasa al siguiente 4. Retorna resultado consolidado
```

### ¿Cómo se usa en Flexygo?

Ejemplo: Asistente principal en Flexygo recibe pregunta. Detecta que necesita un experto especializado. Delega a otro asistente (ej: 'specialist-legal', 'specialist-technical'). El asistente remoto responde, y Flexygo retorna la respuesta al usuario.

### ■ ■ Issues:

- **Control de loops débil:** Solo cuenta asistentes (max 30), no valida grafo de deps.
- **copyAnswer poco clara:** Debería ser nombre explícito como isManualResponse.
- **Sin timeout:** Llamadas anidadas pueden colgar indefinidamente.
- **Sin logging:** No hay auditoría de qué asistentes se llamaron.
- **Ineficiente:** Recrea tool config en cada llamada.
- **Sin rate limiting:** Puede spam asistentes remotos.

## 8. RAGCategory.cs - Categorización de Documentos

Estructura para agrupar documentos en categorías lógicas. Permite filtrar búsquedas por tipo de documento (ej: 'mostrar solo reportes financieros').

### Propiedades:

| Propiedad  | Tipo   | Propósito                                |
|------------|--------|------------------------------------------|
| RagId      | string | ID del RAG                               |
| Descrip    | string | Nombre/descripción de la categoría       |
| NDocuments | int    | Cantidad de documentos en esta categoría |

### Código del archivo:

```
public class RAGCategory { public string RagId { get; set; } public string Descrip {  
get; set; } public int NDocuments { get; set; } }
```

### ¿Cómo se usa en Flexygo?

Cada cliente puede categorizar sus documentos. Ej: 'Reportes' (300 docs), 'Manuales' (150 docs), 'Normativas' (50 docs). Cuando el usuario busca, puede filtrar por categoría, y RAGProcessor solo compara contra esa categoría.

### ■ ■ Issues/Mejoras:

- Es muy básico — sin jerarquía ni tags múltiples.
- NDocuments no se actualiza automáticamente al añadir/borrar docs.
- No hay relación directa entre RAGCategory y RAGDocument en el código.
- Falta metadata: fecha última actualización, versión, descripción larga.

# Resumen Ejecutivo

## Arquitectura actual:

Flexygo RAG es un sistema modular y bien estructurado con separación clara de concerns. Permite a clientes indexar sus propios documentos y hacer búsquedas semánticas sobre ellos.

## Fortalezas:

- ✓ Búsqueda paralela (scalable con múltiples cores)
- ✓ Integración con OpenAI embeddings
- ✓ Fallback a búsqueda web (Google API)
- ✓ Encadenamiento de asistentes
- ✓ Generérico (funciona con cualquier cliente)

## Debilidades críticas:

- ✗ Bug en CosineSimilarity (implementación incorrecta)
- ✗ Búsqueda  $O(n)$  — escalabilidad pobre con muchos documentos
- ✗ Threshold bajo → muchos falsos positivos
- ✗ Sin índices vectoriales (HNSW)
- ✗ Sin caché de búsquedas
- ✗ Playwright lento para web scraping

## Recomendaciones (prioridad):

### 1. Crítico (Corregir bugs):

- Reparar CosineSimilarity en VectorMath
- Normalizar scores a rango 0-1
- Mover threshold y dimensiones a RAGConfig
- Añadir validación de dimensiones en RAGVector

### 2. Alto (Rendimiento):

- Implementar HNSW en SQL Server (si hay muchos docs)
- Añadir caché distribuida (Redis) para embeddings de queries
- Pool de conexiones Playwright
- Timeout en todas operaciones async

### 3. Medio (Mejoras):

- Chunking automático de docs largos
- Logging de métricas de búsqueda
- Endpoints FastAPI para cada componente
- Tests de performance